Adevar Labs

Prepared for:

Lamport Systems, Inc.

December 15, 2025

Protocol Integration Patches Audit Report

Version: Audit report after fix review

Table of Contents

1. Executive Summary **3**

About GLAM 3

Audit Summary 3

Overall Security Posture 3

After Fix Review Summary 3

Before Fix Review Summary 3

Risk Profile 3

Launch Recommendations 4

Audit Scope 4

2. Assumptions and Considerations **5**

3. Severity Definitions **6**

Impact 6

Likelihood 6

Severity Classification Matrix 6

4. Enhancement Opportunities **8**

E01: Enforce more checks on "before" and "after" token snapshots verifications 8

E02: Replicate check on destination_token_account on RouteV2 same as Route 10

About Us **11**

About Adevar Labs 11

Audit Methodology 11

Confidentiality Notice 12

Legal Disclaimer 12

Appendix A: Enforce more checks on "before" and "after" token snapshots verifications POC **14**

1. Executive Summary

About GLAM

GLAM is a protocol for programmable investment infrastructure on Solana. It provides composable primitives like vaults, mints, integrations, policies, and permissions to launch tokenized products and onchain strategies with precision and control.

Audit Summary

This audit reviewed two GLAM pull requests and one commit; conclusions:

- Jupiter swap PR replaces unused v1 with v2 and adds slippage checks.
- Kamino fix PR addresses an upstream breaking change without altering vault accounting.
- sync_native commit auto-syncs wSOL when SOL is transferred into the wrapped account.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	0	0	0

Enhancement opportunities: 2.

Overall Security Posture

No new risks introduced; two defense-in-depth enhancements recommended.

After Fix Review Summary

After-fix review confirmed the correctness of both enhancements' implementations.

Before Fix Review Summary

Before-fix review showed no exploitable issues given the scoped changes.

Risk Profile

No risks identified in scope; two improvements to harden CPI snapshot verification.

Launch Recommendations

Ready to ship, both fixes are verified, snapshot identity is enforced before any additional CPI integrations.

Audit Scope

- **Repository:** https://github.com/glamsystems/glam
- **PRs in scope:**
  - https://github.com/glamsystems/glam/pull/876
  - https://github.com/glamsystems/glam/pull/877
- **Commits in Scope:**
  - https://github.com/glamsystems/glam/commit/44720b28238b0ec8305dd66e55f9db03b25b44ba
  - Tip of the repository at time of review.
- After fix commit hash: 1a8a73c3e329b2de1e25292caff59ed6a65e8d0c

2. Assumptions and Considerations

- Vault delegators use Jupiter swaps in good faith, supplying reasonable quotes.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements–typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue–particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

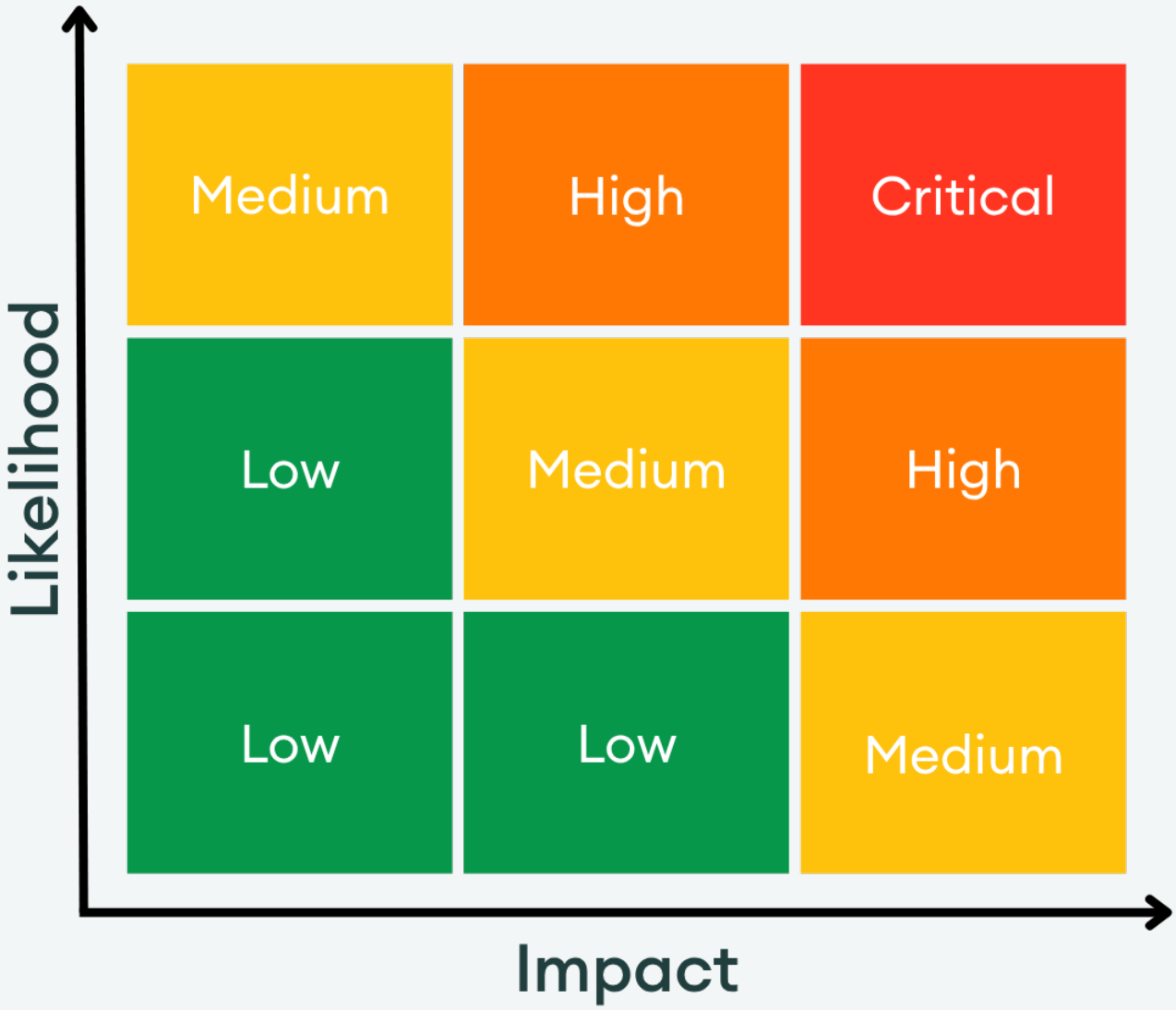
- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substansial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability
- **Low**: Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Enhancement Opportunities

E01: Enforce more checks on "before" and "after" token snapshots verifications

Location: https://github.com/glamsystems/glam/blob/1c1c5ede50d2358c4748d0f6e70efa273e09ff6d/anchor/libs/common/src/macros.rs#L82-L101

>_ macros.rs RUST

```
80:         )?;
81:
82:         // 4. Validate no unauthorized balance decreases
83:         if before.len() != after.len() {
84:             anchor_lang::prelude::msg!(
85:                 "Vault balance check failed: account count mismatch (before: {}, after
        : {})",
86:                 before.len(),
87:                 after.len()
88:             );
89:             return Err($error.into());
90:         }
91:         for i in 0..before.len() {
92:             if after[i] < before[i] {
93:                 anchor_lang::prelude::msg!(
94:                     "Vault balance check failed: unauthorized decrease at index {} (be
        fore: {}, after: {})",
95:                     i,
96:                     before[i],
97:                     after[i]
98:                 );
99:                 return Err($error.into());
100:             }
101:         }
102:     }};
103: }
```

Description:

The code compares the "after" state to the "before" state by ensuring the array lengths match and that no value decreases after the invoke. That only works when the accounts and mints being measured remain the same.

The snapshot helper here skips accounts explicitly (via the skipped param) and implicitly (if the account is not owned by the vault, not a token account, etc.). Because the snapshot only records amounts per index, a CPI target can change the underlying accounts or mints while keeping the captured balances unchanged.

Example with two remaining accounts:

RUST

```
remaining_accounts[0] = vault_ta_100_sol
remaining_accounts[1] = vault_ta_100_usd
[...]
```

The CPI target could transfer out the USD tokens, close vault_ta_100_usd, and reopen it at the same index with a dummy mint:

```rust
remaining_accounts[0] = vault_ta_100_sol
remaining_accounts[1] = vault_ta_100_dummy
[...]
```

Both the "before" and "after" arrays become [100, 100], so the check passes even though the second account now holds a different mint.

Example with three remaining accounts:

```rust
remaining_accounts[0] = vault_ta_100_sol
remaining_accounts[1] = vault_ta_100_usd
remaining_accounts[2] = uninitialized
[...]
```

The CPI target could move the USD tokens out, close vault_ta_100_usd, and open a dummy token account using the uninitialized slot:

```rust
remaining_accounts[0] = vault_ta_100_sol
remaining_accounts[1] = closed
remaining_accounts[2] = vault_ta_100_dummy
[...]
```

Again the "before" and "after" arrays are both [100, 100], but the account order and token mints differ.

Potential Benefit:

Tighter checks make integrations more resilient if a CPI target is compromised or malicious.

Proof of Concept:

See Appendix A.

Recommendation:

Snapshot and compare a tuple of (mint, index, amount), and ideally the account key, so post-invoke verification ensures the same accounts/mints are preserved, not just the balances.

E02: Replicate check on destination_token_account on RouteV2 same as Route

Location: https://github.com/glamsystems/glam/blob/1c1c5ede50d2358c4748d0f6e70efa273e09ff6d/anchor/programs/glam_protocol/src/instructions/jupiter_swap.rs#L188-L192

>_ jupiter_swap.rs RUST

```
186:                 (slippage_bps, platform_fee_bps, 3, 6, 8, 13)
187:             }
188:             RouteV2::DISCRIMINATOR => {
189:                 let (slippage_bps, platform_fee_bps) =
190:                     Jupiter::parse_slippage_and_platform_fee_v2_route(&data);
191:                 (slippage_bps, platform_fee_bps, 1, 2, 4, 10)
192:             }
193:             SharedAccountsRouteV2::DISCRIMINATOR => {
194:                 let (slippage_bps, platform_fee_bps) =
```

Description:

Validate that the destination_token_account is none in RouteV2 similar to how it is verified on RouteV1 path.

Potential Benefit:

Keeps behavior consistent and prevents accidental destination_token_account values in RouteV2.

Recommendation:

Add the same destination_token_account check to the RouteV2 path.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)

- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in

connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

Appendix A: Enforce more checks on "before" and "after" token snapshots verifications POC

RUST

```rust
use anchor_lang::prelude::{AccountInfo, Pubkey};
use anchor_lang::solana_program::{entrypoint::ProgramResult, program_pack::Pack};
use anchor_lang::{InstructionData, ToAccountMetas};
use solana_program_test::{processor, ProgramTest};
use solana_sdk::{
    instruction::Instruction,
    signature::{Keypair, Signer},
    system_instruction,
    transaction::Transaction,
};
use spl_token::{self, state::Account as TokenAccount};

fn process_instruction(
    program_id: &Pubkey,
    accounts: &[AccountInfo],
    instruction_data: &[u8],
) -> ProgramResult {
    // Safety: Solana runtime supplies accounts with a consistent lifetime; this
    // coercion only narrows it so we can call the Anchor entrypoint.
    let accounts: &[AccountInfo<'_>] = unsafe {
        std::mem::transmute::<&[AccountInfo], &[AccountInfo<'_>]>(accounts)
    };
    hello::entry(program_id, accounts, instruction_data).map_err(Into::into)
}


#[tokio::test]
async fn initialize_logs_greeting() {
    solana_logger::setup();

    let program_test =
        ProgramTest::new("hello", hello::ID, processor!(process_instruction));

    let (banks_client, payer, recent_blockhash) = program_test.start().await;

    let ix = Instruction {
        program_id: hello::ID,
        accounts: hello::accounts::Initialize {}.to_account_metas(None),
        data: hello::instruction::Initialize {}.data(),
    };

    let mut tx = Transaction::new_with_payer(&[ix], Some(&payer.pubkey()));
    tx.sign(&[&payer], recent_blockhash);

    banks_client.process_transaction(tx).await.unwrap();
}

#[tokio::test]
async fn reuse_plain_token_account_address() {
    solana_logger::setup();

    let mut program_test =
        ProgramTest::new("hello", hello::ID, processor!(process_instruction));
```

... continued RUST

```rust
    program_test.add_program(
        "spl_token",
        spl_token::id(),
        processor!(spl_token::processor::Processor::process),
    );

    let context = program_test.start_with_context().await;
    let rent = context.banks_client.get_rent().await.unwrap();
    let payer = &context.payer;

    let mint_a = Keypair::new();
    let mint_b = Keypair::new();
    let token_account = Keypair::new();

    let mint_rent = rent.minimum_balance(spl_token::state::Mint::LEN);
    let account_rent = rent.minimum_balance(TokenAccount::LEN);

    let setup_instructions = vec![
        system_instruction::create_account(
            &payer.pubkey(),
            &mint_a.pubkey(),
            mint_rent,
            spl_token::state::Mint::LEN as u64,
            &spl_token::id(),
        ),
        spl_token::instruction::initialize_mint(
            &spl_token::id(),
            &mint_a.pubkey(),
            &payer.pubkey(),
            None,
            0,
        )
        .unwrap(),
        system_instruction::create_account(
            &payer.pubkey(),
            &mint_b.pubkey(),
            mint_rent,
            spl_token::state::Mint::LEN as u64,
            &spl_token::id(),
        ),
        spl_token::instruction::initialize_mint(
            &spl_token::id(),
            &mint_b.pubkey(),
            &payer.pubkey(),
            None,
            0,
        )
        .unwrap(),
        system_instruction::create_account(
            &payer.pubkey(),
            &token_account.pubkey(),
            account_rent,
            TokenAccount::LEN as u64,
            &spl_token::id(),
        ),
        spl_token::instruction::initialize_account3(
            &spl_token::id(),
            &token_account.pubkey(),
```

... continued RUST

```
            &mint_a.pubkey(),
            &payer.pubkey(),
        )
        .unwrap(),
    ];

    let mut setup_tx =
        Transaction::new_with_payer(&setup_instructions, Some(&payer.pubkey()));
    setup_tx.sign(
        &[payer, &mint_a, &mint_b, &token_account],
        context.last_blockhash,
    );
    context
        .banks_client
        .process_transaction(setup_tx)
        .await
        .unwrap();

    let initial = context
        .banks_client
        .get_account(token_account.pubkey())
        .await
        .unwrap()
        .expect("token account should exist");
    let parsed_initial = TokenAccount::unpack(&initial.data).unwrap();
    assert_eq!(parsed_initial.mint, mint_a.pubkey());

    let close_ix = spl_token::instruction::close_account(
        &spl_token::id(),
        &token_account.pubkey(),
        &payer.pubkey(),
        &payer.pubkey(),
        &[],
    )
    .unwrap();
    let top_up_ix = system_instruction::transfer(
        &payer.pubkey(),
        &token_account.pubkey(),
        account_rent,
    );
    let reassign_ix = system_instruction::assign(
        &token_account.pubkey(),
        &spl_token::id(),
    );
    let reinit_ix = spl_token::instruction::initialize_account3(
        &spl_token::id(),
        &token_account.pubkey(),
        &mint_b.pubkey(),
        &payer.pubkey(),
    )
    .unwrap();

    let latest_blockhash = context
        .banks_client
        .get_latest_blockhash()
        .await
        .unwrap();
    let mut reuse_tx = Transaction::new_with_payer(
```

... continued RUST

```
        &[close_ix, top_up_ix, reassign_ix, reinit_ix],
        Some(&payer.pubkey()),
    );
    reuse_tx.sign(&[payer, &token_account], latest_blockhash);
    context
        .banks_client
        .process_transaction(reuse_tx)
        .await
        .unwrap();

    let reopened = context
        .banks_client
        .get_account(token_account.pubkey())
        .await
        .unwrap()
        .expect("token account should be reinitialized");
    let parsed_reopened = TokenAccount::unpack(&reopened.data).unwrap();
    assert_eq!(parsed_reopened.mint, mint_b.pubkey());
    assert_eq!(parsed_reopened.owner, payer.pubkey());
    assert_eq!(reopened.owner, spl_token::id());
    assert!(
        reopened.lamports >= account_rent,
        "account is rent exempt after reinitialization"
    );
}
```